CS-E4740 - Federated Learning '26

Project Kickoff

Assoc. Prof. Alexander Jung

Calendar **Glossary** **Book** **GitHub**

Table of Contents

Learning Goals

Mid-Course Feedback

Generic Project Workflow

Deliverables and Grading

10 ECTS Extension

Next Steps

Table of Contents

Learning Goals

Mid-Course Feedback

Generic Project Workflow

Deliverables and Grading

10 ECTS Extension

Next Steps

Project Learning Goals

1. **Apply FL theory to a practical problem.** Design and implement your own FL system using real FMI weather data.

2. **Write a well-structured conference paper.** Formulate the problem, describe the method, present experiments, and draw conclusions — all within 5 pages.

3. **Give and receive constructive peer feedback.** Review other students' reports and revise your own work based on the feedback you receive — just like the real publication process.

Table of Contents

Learning Goals

Mid-Course Feedback

Generic Project Workflow

Deliverables and Grading

10 ECTS Extension

Next Steps

Feedback Highlights

Respondents: 32 / 106 (30%)

Time spent so far	59%	around 50 h
Workload	75%	as expected
Step 1 (GTVMin formulation)	87%	confident
Step 2 (fixed-point equation)	81%	confident
Step 3 (FL algorithm design)	83%	confident

Least clear: Step 3 (46%), Step 2 (34%)

Kickoff priorities:

Choosing / implementing an FL algorithm	34%
Getting started with FMI data	33%
Grading and peer review	16%
High-quality / publishable report	16%

Table of Contents

Learning Goals

Mid-Course Feedback

Generic Project Workflow

Deliverables and Grading

10 ECTS Extension

Next Steps

FMI Data

Raw data source (fixed for all projects): FMI Open Data

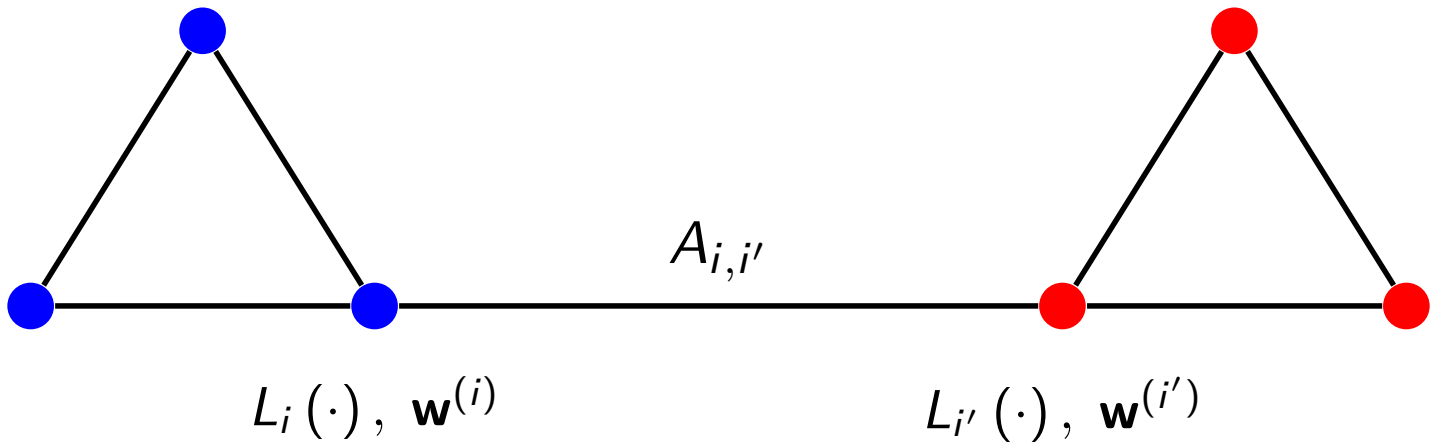
FMI Open Data

`GetFMIData.py`

`GetFMIData.py` queries hourly observations (temperature, humidity, wind speed, ...) per station.

FMI provides the *raw data* only. Everything else is **your design choice**.

From FMI Stations to an FL Network



Fixed: each device $i = 1, \ldots, n$ is a FMI station

Your design choices:

- **Data points:** define features $\mathbf{x}$ and label y from raw FMI data
- **Edges:** how to connect stations and set weights $A_{i,i'}$
- **Model:** $\mathbf{w}^{(i)} \in \mathbb{R}^d$ at each device
- **Loss:** $L_i(\cdot)$ on $\mathcal{D}^{(i)}$ (e.g. MSE)

FL Design Workflow

1. **Formulate** FL application as GTVMin

2. **Characterize** solutions via fixed-point equation:

$$\mathbf{w}^{(i)} \;=\; \mathcal{F}^{(i)}\big(\mathbf{w}^{(1)}, \ldots, \mathbf{w}^{(n)}\big) \qquad \text{for each node } i$$

   where $\mathcal{F}^{(i)}$ is the **block-wise minimization operator** at node i

3. **Iterate** the operator to obtain an FL algorithm:

$$\mathbf{w}^{(i,t+1)} \;=\; \mathcal{F}^{(i)}\big(\mathbf{w}^{(1,t)}, \ldots, \mathbf{w}^{(n,t)}\big) \qquad \text{for } t = 0, 1, 2, \ldots$$

Step 1: Formulate GTVMin

Given the FL network, solve:

$$\min_{\mathbf{w}^{(1)},\ldots,\mathbf{w}^{(n)}} \underbrace{\sum_{i=1}^{n} L_i\left(\mathbf{w}^{(i)}\right)}_{\text{local fit}} + \alpha \underbrace{\sum_{\{i,i'\}\in\mathcal{E}} A_{i,i'} \left\|\mathbf{w}^{(i)} - \mathbf{w}^{(i')}\right\|^2}_{\text{GTV penalty}}$$

- $\alpha = 0$: nodes learn **independently**
- $\alpha \to \infty$: **single shared model**
- intermediate α: **collaboration** $\propto$ **similarity**

Step 2: Fixed-Point Equation

Optimality condition for squared-ℓ_2 variation:

$$\mathbf{w}^{(i)} \;=\; \mathcal{F}^{(i)}\big(\mathbf{w}^{(1)}, \ldots, \mathbf{w}^{(n)}\big)$$

with

$$\mathcal{F}^{(i)} \;:=\; \mathbf{w}^{(i)} - \eta\Big[\nabla L_i\left(\mathbf{w}^{(i)}\right) + \alpha \sum_{i' \in \mathcal{N}^{(i)}} A_{i,i'}\left(\mathbf{w}^{(i)} - \mathbf{w}^{(i')}\right)\Big]$$

Balances local fit against agreement with neighbours.

Step 3: Iterate the Fixed-Point Operator

Initialize $\mathbf{w}^{(i,0)}$; for $t = 0, 1, 2, \ldots$ each node computes:

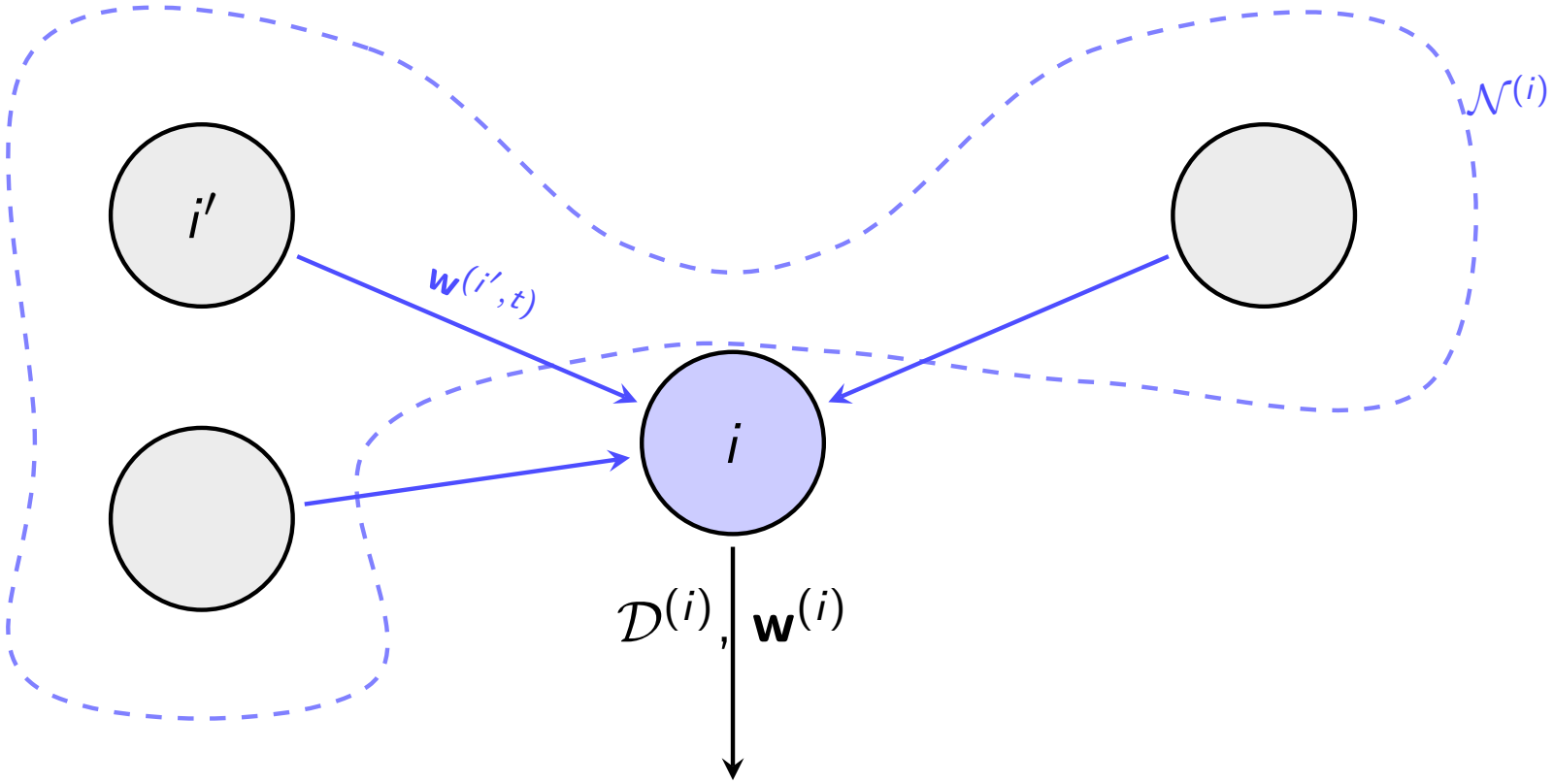
$$\mathbf{w}^{(i,t+1)} = \mathbf{w}^{(i,t)} - \eta \Big[\nabla L_i \left(\mathbf{w}^{(i,t)} \right) + \alpha \sum_{i' \in \mathcal{N}^{(i)}} A_{i,i'} \left(\mathbf{w}^{(i,t)} - \mathbf{w}^{(i',t)} \right) \Big]$$

Step 3: Python Implementation at Node i

```
# local gradient on node i's own data
grad_local = compute_gradient(X_i, y_i, w_i)

# gather current parameters from neighbours
w_neighbours = {i_prime: receive_params(i_prime)
                for i_prime in neighbours_i}

# weighted disagreement with neighbours
neighbour_diff = sum(A[i,i_prime] * (w_i - w_neighbours[
    i_prime])
            for i_prime in neighbours_i)

# parameter update
w_i = w_i - lr * (grad_local + lam * neighbour_diff)
```

receive_params(i_prime) fetches $\mathbf{w}^{(i')}$ from neighbour i' – no raw data shared.

Step 3: Information Flow at Node i



$$\mathbf{w}^{(i,t+1)} = \mathbf{w}^{(i,t)} - \eta\left[\nabla L_i\left(\mathbf{w}^{(i,t)}\right) + \alpha \sum_{i'} A_{i,i'}\left(\mathbf{w}^{(i,t)} - \mathbf{w}^{(i',t)}\right)\right]$$

Node i uses only its **local data** $\mathcal{D}^{(i)}$ and **model parameters** $\mathbf{w}^{(i')}$ received from neighbours $i' \in \mathcal{N}^{(i)}$.

Sanity Checks

1. **Baseline.** Local-only training ($\alpha = 0$) – your FL method should match or improve on this.
2. **Loss curves.** Training loss should decrease monotonically (convex losses).
3. **Predictions vs. actuals.** Scatter plot for a few stations; points near the diagonal.
4. **Effect of α.** Small $\alpha \approx$ local-only; large $\alpha \approx$ shared model.
5. **Compare ≥ 2 systems.** Differ in a meaningful design choice (graph, model, α, ...).

Table of Contents

Learning Goals

Mid-Course Feedback

Generic Project Workflow

Deliverables and Grading

10 ECTS Extension

Next Steps

Schedule

Milestone	Date	What you do
Kickoff	Wed 01.04	
1st submission	Sun 26.04	report (PDF) + code (zip)
Peer Review 1	27.04 – 03.05	review assigned drafts
2nd submission	Sun 17.05	revised report, response letter, pre-recorded talk (via MyCourses)
Peer Review 2	18.05 – 27.05	review improved reports

Grading

Component	**Points**
First submission (pass/fail)	5
Peer Review 1	4
Second submission	43
Recorded talk (pass/fail, graded by TAs)	–
Peer Review 2	8
Total	**60**

Note: All components (including the recorded talk) are **mandatory** to pass.

Peer review points are scaled from the review sheet to the above maxima.

What Earns Points

- **Problem formulation.** nodes, edges, data points, loss function defined and justified
- **Methodology.** correct GTVMin and algorithm
- **Experiments.** at least 2 FL systems compared
- **Writing.** concise, follows template
- **Reproducibility.** code runs, reproduces results

Report Template and Formatting

Provided in `FLProject26/`:

- `FLReportTemplate_26.tex` – starting point (remove hints before submission)
- `FLReportSample.pdf` – sample report
- `FLProjectReviewSheet.pdf` – peer review rubric

Formatting rules:

- Max 5 pages (5th page = references only)
- Anonymous: DO NOT include names, GitHub links, etc.
- Try to use terminology and notation of `AaltoDictionaryofML.github.io`
- Do not modify section titles or margins
- Code in Python (`.py` or `.ipynb`)

Using LLMs in Peer Review

LLMs (ChatGPT, Claude, ...) may assist your review. Two conditions:

1. **Read the report first.** Form your own assessment before using an LLM. You may be asked to explain your review.
2. **Do not upload private information to an LLM.**

Good uses: check equations, look up methods, improve your review text.

Not acceptable: paste entire report $\rightarrow$ submit raw LLM output.

Table of Contents

Learning Goals

Mid-Course Feedback

Generic Project Workflow

Deliverables and Grading

10 ECTS Extension

Next Steps

10 ECTS – Research Project Extension

- For students aiming at a publishable report (grade ≥ 4)
- Extended scope: novel method, larger experiments, extra datasets
- Conference-paper-quality report + individual supervision
- Discuss scope **as early as possible**; work continues over summer

Interested? Email with your project idea and background.

Table of Contents

Learning Goals

Mid-Course Feedback

Generic Project Workflow

Deliverables and Grading

10 ECTS Extension

Next Steps

Project Garages (Online Drop-In Sessions)

Two online help sessions during the regular lecture slots:

Session	**Date / Time**	**Focus**
Garage 1	Wed 08.04, 16:15	Step 1: GTVMin formulation, getting started with FMI data
Garage 2	Wed 15.04, 16:15	Steps 2–3: implementation, debugging, sanity checks

Format: bring your questions, code, and errors — we help you get unstuck.

Next Steps

1. **Today:** fetch FMI data, define your FL network
2. **This week:** formulate GTVMin (Step 1)
3. **Wed 08.04:** Project Garage 1 (online, 16:15)
4. **Wed 15.04:** Project Garage 2 (online, 16:15)
5. **Sun 26.04:** first submission deadline

Resources

- **Report template + sample:** `Edition2026/FLProject26/` on GitHub
- **FMI data script:** `Edition2026/assets/GetFMIData.py`
- **FLBook:** Ch. 4–7 for GTVMin theory and algorithms
- **Notation + figures:** Aalto Dictionary of ML repo – `assets/ml_macros.tex` (381 macros) and 300+ reusable TikZ figures

GitHub: `Edition2026/`

GitHub: Aalto Dictionary of ML